

Stevens Institute of Technology

Master Of Science in Computer Science

CS590 Algorithm

Name : Goti Parth Rameshbhai

CWID : 20005769

Hw1 : Assignment_Report_1

Question - 1: Develop an improved implementation of insertion sort for integer vector (*insertion_sort_im*) that precomputes the length of each vector before the sorting. Keep in mind that the vectors are sorted according to their length (see *ivector_length* function). You can test the correctness of your sorting algorithm using the provided *check_sorted* function.

Answer:

An improved implementation of insertion sort for integer vector is created in function called *improved_insertion_sort* which would precompute length of integer vectors using for loop. In previously given algorithm length of vector array was computed all time it entered loop before swapping with previous integer arrays which led into increase in complexity to $O(n^3)$. So, developing the efficient led into increase in space of 'm' but which can be compensated because of huge decrease in time complexity by 1 to $O(n^2)$. The algorithm works the same but 'ls' variable is array of 'm' which is used to store the values of length of each array in vector.

Question - 2: Implement a merge sort for an array of integer vectors. As for the improved insertion sort algorithm, you should precompute the length of the vectors before the sorting and the sorting is done according to the vector length. Test the correctness of your merge sort implementation using the provided *check_sorted* function.

Answer:

'merge_sort' function is called from 'main' to implement the 'merge sort' algorithm. In 'merge_sort' function precomputes the vector length in the same previous variable 'ls'. The 'ls' being array is passed to function called 'merge_sort_2' where the recursive implementation of merge sort takes place using divide and conquer and combine way. The vector is divided in this function by computing middle value of the array. The middle value is half of first index and last index. And the 'merge' function is used to 'merge' the divided array by adding values in the 'ascending order'. The time complexity for this algorithm to sort the array of integer vector is $O(n \log 2n)$.

Question - 3: Measure the runtime performance of insertion sort (naive and improved) and merge sort for random, sorted, and inverse sorted inputs of size $m = 1000; 2500; 5000; 10000; 25000; 50000; 100000; 250000$ and vector dimension $n = 10; 25; 50$. You can use the provided functions *create_random_ivector*, *create_sorted_ivector*, *create_reverse_sorted_ivector*. Repeat each test a number of times (usually at least 10 times) and compute the average running time for each combination of algorithm, input, size m , and vector dimension n . Report and comment on your results.

Answer:

I had run results for random inputs with input size from 1000 to 25000 with dimension of 10 to 50 and size 50000 with dimension of 10 and 25, As for size 50000 and dimension 50 or more input size the time required for naïve insertion was more than 5 minutes.

I had run results for sorted inputs with input size from 2500 to 250000 with all dimensions, As for size 1000 with all dimension the time required for all type was less than 0 milliseconds.

I had run results for reverse sorted inputs with input size from 1000 to 25000 with dimension of 10 to 50 and with size 50000 with dimension of 10, As for size 50000 and dimension 25 or more input size the time required for naïve insertion was more than 5 minutes.

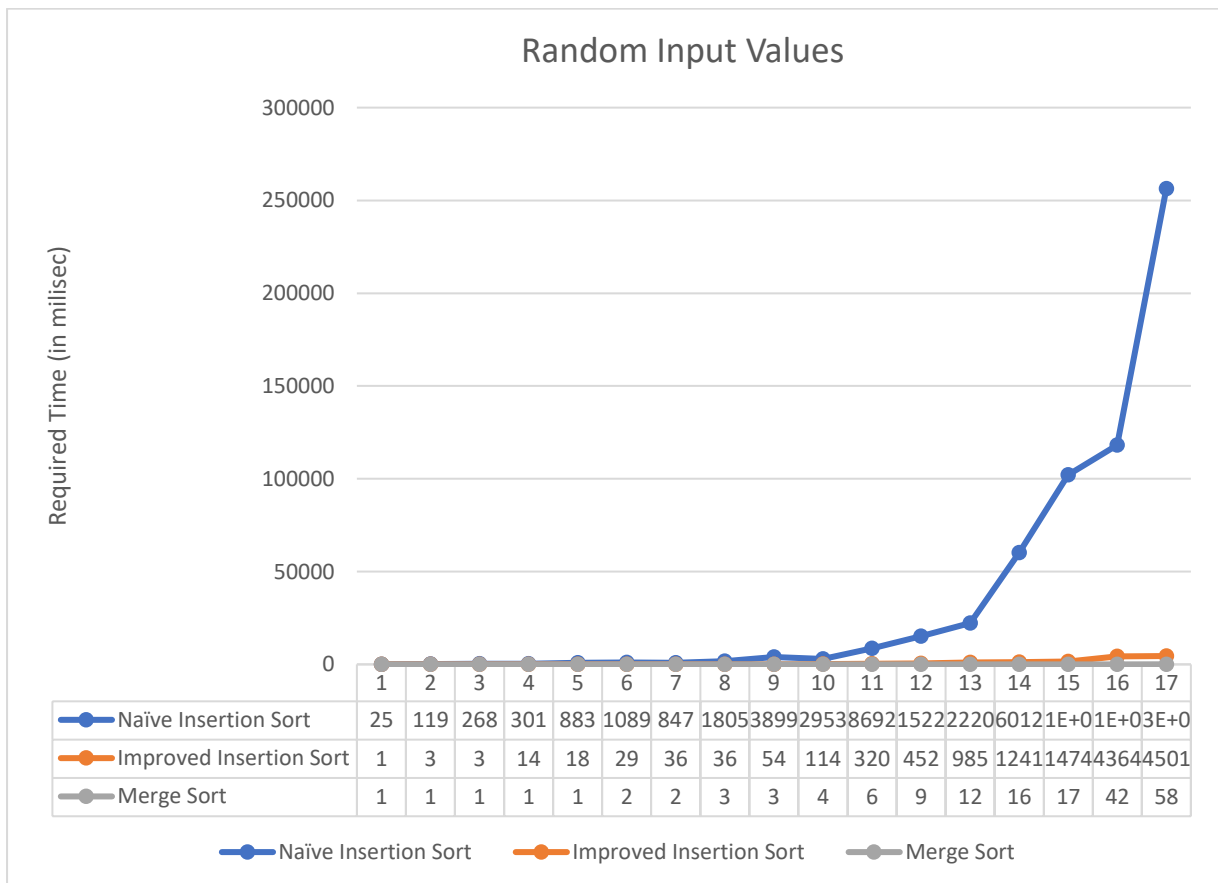
Conclusion:

Random Input Values

Naïve Insertion Sort		
Number of Array (n)	Dimension of Array (m)	Required Time (in millisecond)
1000	10	25
	25	119
	50	268
2500	10	301
	25	883
	50	1089
5000	10	847
	25	1805
	50	3899
10000	10	2953
	25	8692
	50	15226
25000	10	22201
	25	60129
	50	102175
50000	10	118123
	25	256400

Improved Insertion Sort		
Number of Array (n)	Dimension of Array (m)	Required Time (in millisecond)
1000	10	1
	25	3
	50	3
2500	10	14
	25	18
	50	29
5000	10	36
	25	36
	50	54
10000	10	114
	25	320
	50	452
25000	10	985
	25	1241
	50	1474
50000	10	4364
	25	4501

Merge Sort		
Number of Array (n)	Dimension of Array (m)	Required Time (in millisecond)
1000	10	1
	25	1
	50	1
2500	10	1
	25	1
	50	2
5000	10	2
	25	3
	50	3
10000	10	4
	25	6
	50	9
25000	10	12
	25	16
	50	17
50000	10	42
	25	58



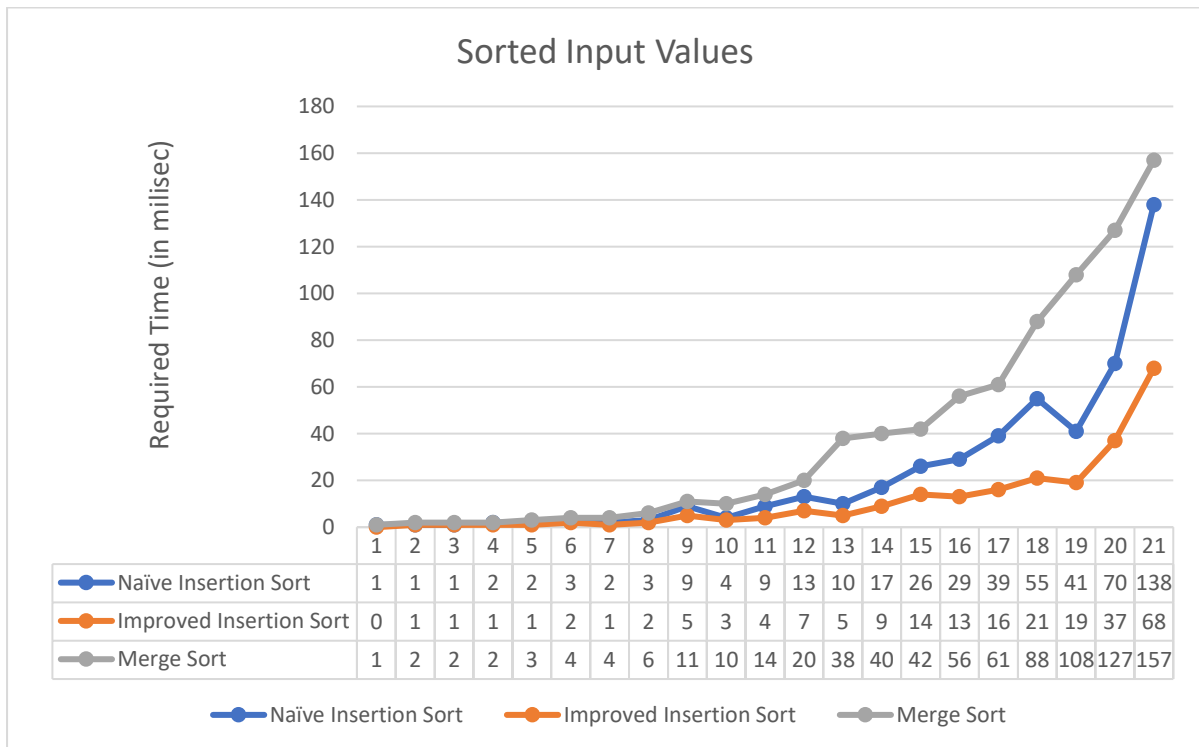
Here, For Random Input values, we can see that Merge Sort is fastest of all for every dimension and height input. Merge Sort is about 78 times faster than Improved Insertion Sort and Improved Insertion Sort is about 57 times faster than Naïve Insertion Sort. The time required for Random input is not highest of all types of input for Naïve and Improved.

Sorted Input Values

Naïve Insertion Sort		
Number of Array (n)	Dimension of Array (m)	Required Time (in millisecond)
2500	10	1
	25	1
	50	1
5000	10	2
	25	2
	50	3
10000	10	2
	25	3
	50	9
25000	10	4
	25	9
	50	13
50000	10	10
	25	17
	50	26
100000	10	29
	25	39
	50	55
250000	10	41
	25	70
	50	138

Improved Insertion Sort		
Number of Array (n)	Dimension of Array (m)	Required Time (in millisecond)
2500	10	0
	25	1
	50	1
5000	10	1
	25	1
	50	2
10000	10	1
	25	2
	50	5
25000	10	3
	25	4
	50	7
50000	10	5
	25	9
	50	14
100000	10	13
	25	16
	50	21
250000	10	19
	25	37
	50	68

Merge Sort		
Number of Array (n)	Dimension of Array (m)	Required Time (in millisecond)
2500	10	1
	25	2
	50	2
5000	10	2
	25	3
	50	4
10000	10	4
	25	6
	50	11
25000	10	10
	25	14
	50	20
50000	10	38
	25	40
	50	42
100000	10	56
	25	61
	50	88
250000	10	108
	25	127
	50	157



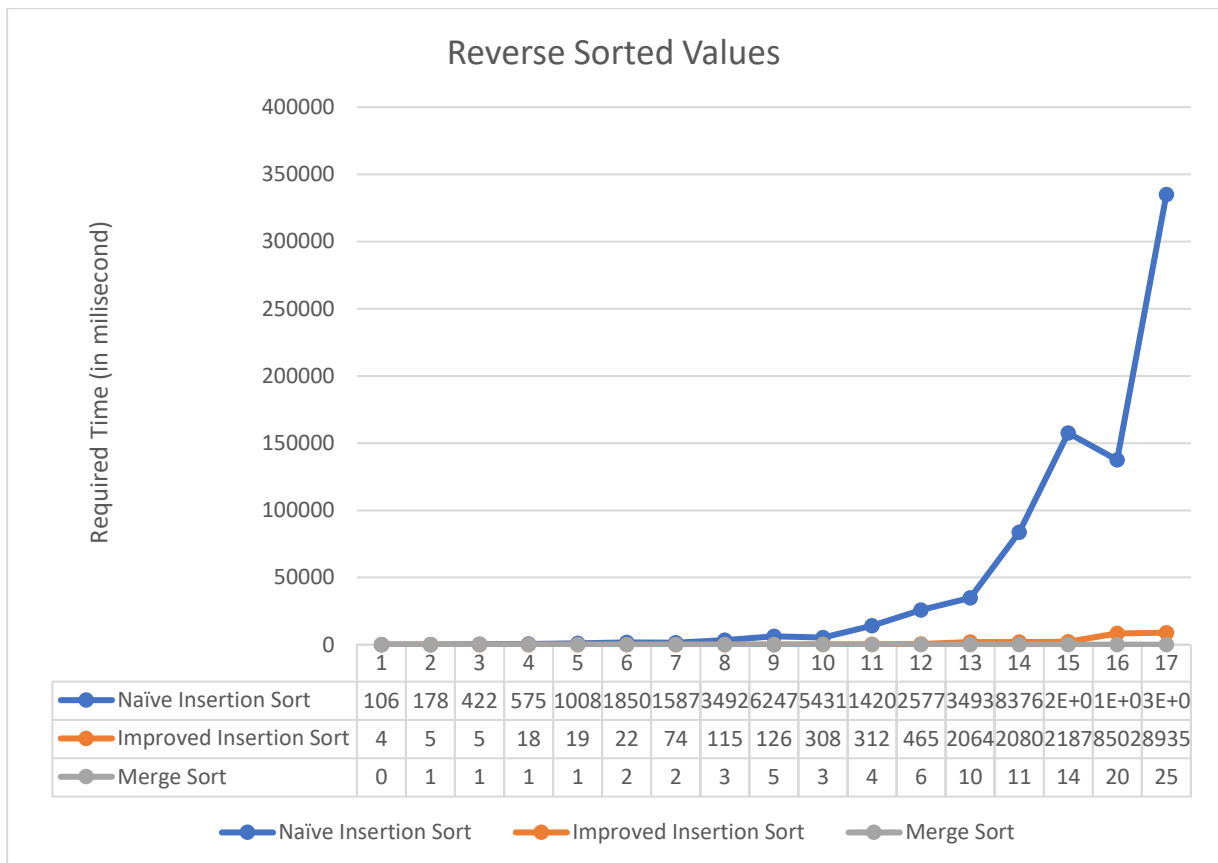
Here, for sorted input values take least time for Naïve Insertion sort and Improved Insertion Sort. Whereas, merge sort in sorted input takes same time more or less as reverse sort as the whether the array is sorted or not, merge sort will divide an array to least dividable value like for example 1 unit. But for small values merge sort works very badly and both naïve and improved works really very well as they don't need process the swapping, just need to run the loop.

Reverse Sorted Input Values

Naïve Insertion Sort		
Number of Array (n)	Dimension of Array (m)	Required Time (in millisecond)
1000	10	106
	25	178
	50	422
2500	10	575
	25	1008
	50	1850
5000	10	1587
	25	3492
	50	6247
10000	10	5431
	25	14201
	50	25779
25000	10	34930
	25	83761
	50	157515
50000	10	137436
	25	334896

Improved Insertion Sort		
Number of Array (n)	Dimension of Array (m)	Required Time (in millisecond)
1000	10	4
	25	5
	50	5
2500	10	18
	25	19
	50	22
5000	10	74
	25	115
	50	126
10000	10	308
	25	312
	50	465
25000	10	2064
	25	2080
	50	2187
50000	10	8502
	25	8935

Merge Sort		
Number of Array (n)	Dimension of Array (m)	Required Time (in millisecond)
1000	10	0
	25	1
	50	1
2500	10	1
	25	1
	50	2
5000	10	2
	25	3
	50	5
10000	10	3
	25	4
	50	6
25000	10	10
	25	11
	50	14
50000	10	20
	25	25



Here, in reverse sorted input values takes most time for Naïve and Improved Sort because, each value has to be swapped to the last position as they are reversely sorted. But merge runs efficiently that case scenario as it doesn't matter which way input are provided.